

Assignment 5: GatherUp — Event Management API

Pending



Objective

Build a **FastAPI-based Event Management API** where users can create events, RSVP to events, view attendees, and cancel events.

The assignment should demonstrate:

- Async Database CRUD
- Async SQLAlchemy or `databases`
- Regex validation
- Date validation
- Capacity management
- User-specific authorization
- Proper HTTP status codes

Technical Requirements

- **Framework:** FastAPI
- Use **Async SQLAlchemy** or the **databases** library
- Event status:
 - upcoming
 - completed
 - cancelled
- Event date must be a **future date**
- Email invites must use **regex validation**
- Creation APIs should return:

201 Created

Data Models

User

- id
- email

Event

- id
- title
- description
- event_date
- max_attendees
- creator_id

RSVP

- id
 - user_id
 - event_id
 - status
-

API Routes

POST /events

Create a new event.

Required fields:

- Title
- Description

- Event date
- Maximum attendees

Validation:

- `event_date` must be in the future
- `max_attendees` must be greater than 0

Successful creation should return:

201 Created

GET /events

List all upcoming events.

Only events where:

`event_date > current date`

should be returned.

Cancelled events should not appear in the upcoming events list.

POST /events/{id}/rsvp

Register the authenticated user for an event.

Rules:

- RSVP must fail if the event is already full.
- Duplicate RSVP from the same user is not allowed.
- RSVP should not be allowed for cancelled events.

If the maximum attendee limit has been reached, return:

400 Bad Request

GET /events/{id}/attendees

Return all users who have successfully registered for the selected event.

The response should include the attendee details associated with that event.

DELETE /events/{id}

Cancel an event.

Requirements:

- Only the user who created the event can cancel it.
 - Other users must not be allowed to cancel the event.
 - The event should be marked as `cancelled`.
-

Email Regex Validation

Email addresses used for invitations must be validated using an appropriate regular expression or Pydantic validation.

Invalid email addresses should be rejected.

Async Database Operations

Database operations must be implemented asynchronously.

Use:

```
async def
```

for database-related route operations wherever appropriate.

Students may use:

- Async SQLAlchemy
 - `databases` library
-

Validation & Business Rules

- Event date must be a future date.
- Maximum attendees must be greater than `0`.
- RSVP must be blocked when the event is full.

- Duplicate RSVP is not allowed.
 - Only the event creator can cancel an event.
 - Cancelled events should not accept new RSVPs.
 - Email invitations must pass validation.
-

Error Handling

Handle appropriate errors for:

- Event not found
- Event already full
- Duplicate RSVP
- Invalid event date
- Invalid email
- Unauthorized event cancellation
- RSVP to cancelled event

Use appropriate HTTP status codes such as 400, 403, 404, 201, and 422.

API Testing

All APIs should be testable through FastAPI Swagger:

/docs

Test:

- Event creation
- Future-date validation
- Upcoming event listing
- RSVP creation
- Maximum capacity validation
- Duplicate RSVP validation

- Attendee listing
 - Creator-only event cancellation
 - Email validation
 - Async CRUD operations
-

Submission Instructions

Submit the **complete project as ONE ZIP file through the LMS.**

The ZIP file should include:

- Complete source code
- `requirements.txt`
- `README.md`
- All files required to run the project

ZIP File Name

`StudentName_GatherUp.zip`

Example:

`RahulKumar_GatherUp.zip`

Important

- Submit only **one ZIP file**.
 - Do not upload individual files separately.
 - Do not include `venv`, `.venv`, `__pycache__`, or unnecessary folders.
 - Test the application before submission.
 - Ensure the correct ZIP file is uploaded successfully to the LMS.
 - Submit before the deadline mentioned in the LMS.
-

Evaluation Criteria

The assignment will be evaluated based on:

- Correct API functionality
- Async database implementation
- Event date validation
- Email validation
- RSVP and capacity logic
- Duplicate RSVP handling
- Creator-only cancellation
- Proper HTTP status codes
- Error handling
- Code quality and organization
- Successful project execution
- Proper ZIP submission

Expected Outcome

By completing this assignment, you should be able to build a structured **FastAPI Event Management API using async database operations, validation, RSVP capacity control, authorization, and proper HTTP status handling.**

Upload Assignment